Int. J. Electron. Commun. (AEÜ) 200 (2025) 155870

Contents lists available at ScienceDirect

Int. J. Electron. Commun. (AEÜ)

journal homepage: www.elsevier.com/locate/aeue

TimeBoost: Accelerating timing analysis engines using XGBoost

Anastasis Vagenas *, Dimitrios Garyfallou, Nestor Evmorfopoulos, George Stamoulis

Department of Electrical and Computer Engineering, University of Thessaly, Volos, Greece

ARTICLE INFO

Keywords:
Static timing analysis
Timing optimization
Gate models
Effective capacitance
Interconnect
Machine learning
Classification
XGBoost
Graphics processing unit

ABSTRACT

Timing analysis is an essential verification method for the design and optimization of integrated circuits, while it also constitutes the cornerstone of the final signoff. However, the increasing complexity of modern designs, driven by aggressive technology scaling, necessitates more efficient timing engines. In this article, we introduce TimeBoost, a novel learning-based approach for performance optimization of gate-level timing analysis. TimeBoost employs GPU-accelerated XGBoost classifiers to evaluate stage complexity and select the optimal timing analysis method, exploiting the available runtime and accuracy trade-off. Unlike prior works that predict timing directly, our approach ensures reliable and accurate timing estimation by leveraging timing analysis methods as prediction targets. Moreover, our models utilize representative electrical, topological, and gate features for each stage, which can be extracted with minimal effort. We assess several Machine Learning (ML) models, including XGBoost, Artificial Neural Networks, and K-Nearest Neighbors, and explore their different hyperparameters. Experimental evaluation on one million stages demonstrates that our ML-based approach achieves a runtime improvement of up to 3.86× over our best available method, imposing only a 3%–4% overhead to timing analysis while maintaining high accuracy. Thus, the proposed models can be integrated into signoff tools or timing-driven iterative optimization flows to expedite timing closure.

1. Introduction

Timing analysis is a critical and demanding process in the design and verification of Very Large Scale Integrated (VLSI) circuits, which ensures that design timing constraints (e.g., setup and hold) are met. It plays a key role in iterative optimization flows, where a timing engine is repeatedly invoked at each step to verify and maintain timing integrity, while it also serves as the cornerstone of the final signoff that determines whether a chip is ready for fabrication [1]. As VLSI designs grow in complexity, incorporating billions of transistors and employing hundreds of analysis scenarios, timing analysis has emerged as the major bottleneck in achieving design closure, accounting for up to 60% of the total design time [2]. Moreover, as technologies scale below 7 nm, increased wire resistivity, signal non-linearities, the Miller effect, and strong gate-interconnect interdependencies render accurate timing analysis even more challenging [3–5]. Therefore, the efficiency and accuracy of a timing engine are crucial to achieving timing closure within reasonable turnaround times. In practice, both static [1] and dynamic [6] timing analysis is typically performed at the stage level, where each stage is composed of a driver gate, a distributed RC interconnect, and a single or multiple receiver gates. Stage timing analysis is further divided into two distinct but interdependent parts, namely gate and interconnect analysis.

Industrial gate models, such as the Non-Linear Delay Model (NLDM) and the more detailed Composite Current Source (CCS) model [7], are utilized to compute the driver output waveform and estimate gate timing. However, these models require the approximation of the distributed RC interconnect as a lumped capacitive load. The simplest load model is the sum of all interconnect capacitance values (C_{total}), while more accurate approaches involve the calculation of an effective capacitance (C_{eff}) [3,4,8,9] at the cost of additional runtime and memory overhead. On the other hand, the golden standard for interconnect analysis is SPICE simulation [10,11], but it is prohibitive due to its computational demands. To address this challenge, Model-Order Reduction (MOR) techniques [12–14] are commonly employed to create a Reduced-Order Model (ROM) of the interconnect, enabling more efficient simulation compared to using the original RC circuit. The most well-known methods, AWE and RICE [13,14], match the first few moments of the interconnect transfer function and create a ROM to approximate its transient behavior. Although both techniques can be highly accurate, they require significant computations and are prone to numerical instability, especially when using higher-order approximations. Instead, faster analytical moment-based [15–18] or C_{eff}-based [4,9] interconnect delay and slew metrics are typically

* Corresponding author.
E-mail addresses: avagenas@e-ce.uth.gr (A. Vagenas), digaryfa@e-ce.uth.gr (D. Garyfallou), nestevmo@e-ce.uth.gr (N. Evmorfopoulos), georges@e-ce.uth.gr (G. Stamoulis).

https://doi.org/10.1016/j.aeue.2025.155870
Received 5 January 2025; Accepted 17 May 2025
Available online 9 June 2025
1434-8411/© 2025 The Authors. Published by Elsevier GmbH. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

utilized. Therefore, selecting a suitable combination of gate and load model, as well as interconnect analysis method, is crucial for balancing the accuracy and runtime of gate-level timing analysis.

Recent efforts to accelerate timing analysis harness parallel computing, such as CPU multi-threading, GPUs, and CPU–GPU heterogeneous computing, exploiting the available parallelism within timing analysis tasks [19]. For instance, enhancing the efficiency of Multi-Corner Multi-Mode (MMCM) analysis [20], Path-Based Analysis (PBA) [21], or delay calculation [20–22] can lead to speedups of up to three orders of magnitude. However, such approaches are closely tied to the underlying timing analysis method, requiring significant modifications to the engine's data structures and algorithms. Another mainstream approach for improving accuracy and efficiency is Machine Learning (ML) [23, 24], which has been extensively employed in characterization [25,26], modeling of complex effects [27,28], and timing estimation [4,29–32]. More specifically, the availability of scalable frameworks [33–35] and the ability of learning-based models to represent multi-dimensional data and complex relationships is particularly valuable for interconnect [29–31] and stage [4,32] timing estimation. The initial approach in [29] attempts to reduce the number of expensive signoff tool invocations by utilizing a linear regression model with several moment-based metrics as inputs. The authors in [30] used Random Forest regressors to estimate pre-routing timing and explored different features. In [31], Graph-Neural Networks (GNN) were used to accurately predict interconnect delay during signoff, considering interconnect topology and RC characteristics. Nevertheless, these works [29–31] focus only on interconnect timing, assuming accurate gate timing, and require the extraction and computation of multiple features before inference, such as moment-based metrics [29], driver and receiver locations [30], or interconnect topology and RC values [31]. To sidestep these limitations, [4, 32] focus on predicting stage timing, considering gate-interconnect interdependencies through their input features. The winners of the ACM TAU 2021 contest [4,36] leverage an efficient algorithm to extract stage features and provide initial stage timing estimates, and then employ ANNs to correlate with golden SPICE results. Similarly, [32] proposes a custom ANN model, which integrates Transformers and Residual Networks (ResNet), to estimate stage delay using multiple moment-based and C_{eff}-based features. A critical issue of most ML-based timing analysis methods is that they make direct predictions [4,29–32], which might be unreliable when the features deviate from the model's training data space. Furthermore, they require large and diverse datasets that are often difficult to obtain, while the training process may demand considerable computational resources. Thus, although ML is a powerful tool for timing prediction, ensuring reliability is a persistent challenge. Finally, the authors in [37] utilized two filtering techniques to choose among different gate and interconnect algorithms. However, these techniques are applied independently, disregarding gate-interconnect interdependencies, while they assume pi-model loads and rely on heuristic moment-based thresholds.

In this article, we introduce TimeBoost, a novel ML-based approach to enhance the efficiency of gate-level timing analysis, which is an extended version of our previous work [38]. We employ GPU-accelerated XGBoost classifiers to analyze stage complexity and select the optimal timing analysis method. By opting for simpler methods in low-complexity stages, we can improve the efficiency of timing analysis without compromising accuracy. Unlike related works that predict timing directly, TimeBoost ensures reliable and accurate estimation. Moreover, it utilizes representative electrical and topological stage features, avoiding unnecessary computations such as C_{eff}. To achieve this, we assess several ML models, including XGBoost (XGB), Artificial Neural Networks (ANN), and K-Nearest Neighbors (KNN), and explore their different hyperparameters. Experimental results indicate that our XGBoost models achieve a runtime improvement of up to 3.86× over our best available method, introducing an overall overhead of only 3%–4% to timing analysis while maintaining high accuracy. Thus, the proposed models can be integrated into signoff tools or timing-driven iterative optimization flows to expedite timing closure.

The rest of this article is organized as follows. Section 2 provides the necessary background material on gate and interconnect timing analysis. In Section 3, we introduce our ML-based approach for performance optimization of timing analysis and evaluate different models and their respective hyperparameters. Section 4 presents our experimental results, while Section 5 concludes this work.

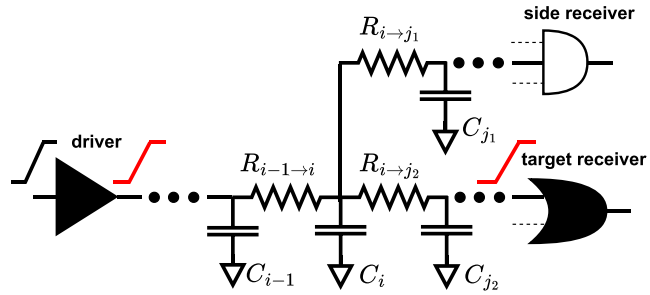


Fig. 1. A gate-level stage with multiple fanout receivers.

2. Background

Fig. 1 illustrates a gate-level stage composed of a driver gate, a distributed RC interconnect, and multiple receiver gates. Each node i of the RC-tree interconnect has a capacitance to ground C_i, a downstream capacitance C_{tot_i} (i.e., the sum of all the capacitances of the subtree starting at node i), a slew value S_i, and is connected to each fanout node j via a resistance $R_{i\to j}$. The sum of all interconnect $R_{i\to j}$ values is denoted as R_{total}.

During stage timing estimation, gate analysis calculates the driver's output voltage waveform to determine the driver delay and slew, while interconnect analysis estimates the target receiver's slew and interconnect delay. Together, these values yield the overall stage delay (i.e., the sum of the driver and interconnect delay) and slew (i.e., target receiver slew). Then, the computed stage delay is utilized to estimate the overall delay across various timing paths in the circuit, while the stage slew is used as input to the next gate-level stage [1].

2.1. Gate timing

Library-compatible gate models, such as NLDM and CCS [3], capture the timing and input capacitance information of each gate into Look-Up Tables (LUTs) indexed by input slew and output load. These pre-characterized LUTs are used by gate-level timing analysis engines for accurate stage delay and slew estimation. A brief overview of the industrial NLDM and CCS models is provided below.

- **NLDM driver model** captures the driver output slew/delay as a function of driver input slew ($S^{\text{in}}_{\text{drv}}$) and output load ($C^{\text{out}}_{\text{drv}}$). Only a single value is used for slew/delay modeling; hence, the non-linearities of the driver output waveform cannot be effectively modeled. As shown in Fig. 2, this information is stored in `cell_rise/fall` and `rise/fall_transition` LUTs for gate delay and slew, respectively.
- **NLDM receiver model** uses a single input pin capacitance (C_p) value to model the non-linear receiver transistor input capacitance. As depicted in Fig. 2, the NLDM receiver capacitance value is stored in the `rise/fall_capacitance` fields of each standard cell. Unfortunately, this model cannot accurately capture the Miller effect or the dependency of C_p to the input slew ($S^{\text{in}}_{\text{rcv}}$) and the output load ($C^{\text{out}}_{\text{rcv}}$) of the receiver cell.
- **CCS driver model** captures the non-linear driver output current waveform as a function of driver input slew ($S^{\text{in}}_{\text{drv}}$) and output load ($C^{\text{out}}_{\text{drv}}$). As shown in Fig. 2, this information is stored in `output_current_rise/fall` LUTs. The time instant when

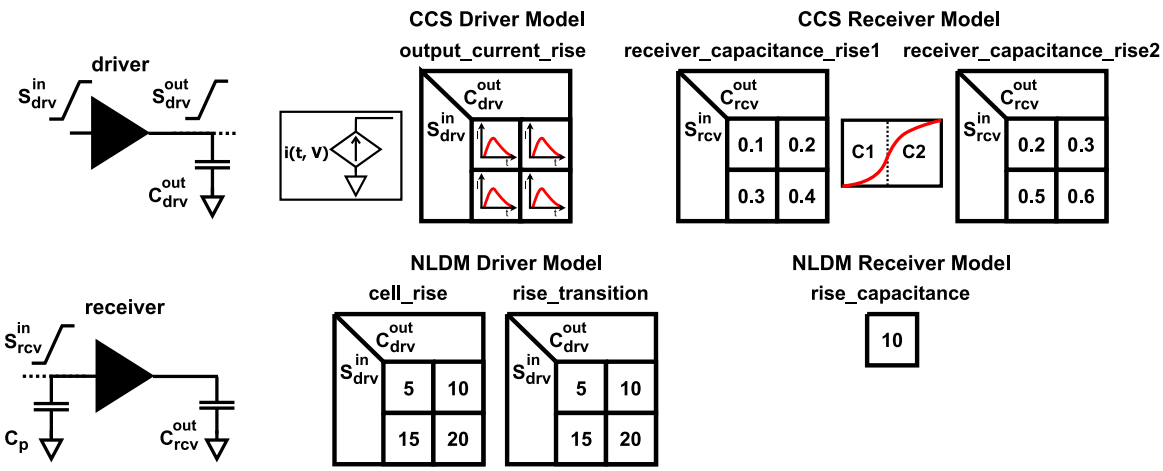


Fig. 2. NLDM and CCS driver and receiver models.

the corresponding driver input voltage crosses the delay threshold (usually $0.5V_{dd}$), which is necessary to calculate gate delay, is also stored as `reference_time` in these LUTs.

- **CCS receiver model** typically uses two different C_p values, $C1$ and $C2$, to model the non-linear receiver transistor input capacitance and the Miller effect. This is very important since the driver load depends both on the RC interconnect and the input pin capacitance of multiple receiver gates. $C1$ is considered up to the delay threshold of the receiver input waveform, while $C2$ is considered past this point. More than two regions and values can be used to improve accuracy. As depicted in Fig. 2, CCS receiver capacitance values are stored in `receiver_capacitance_1/2_rise/fall` LUTs and are also dependent on S^{in}_{rcv} and C^{out}_{rcv} of the receiver cell.

2.1.1. Modeling the RC interconnect load

During standard cell characterization, a lumped capacitive load is typically assumed [39,40]. Thus, a single capacitance value must be used to represent the highly resistive RC interconnect of each stage and access the LUTs. The simplest and most straightforward approximation of an RC interconnect load is C_{total}, which ignores the resistive shielding effect of complex on-chip interconnects, resulting in increased pessimism during gate timing analysis [3].

To overcome the limitations of C_{total}, the concept of C_{eff} is often utilized [3,4,8,9], which is an equivalent capacitance that can be used as a load to approximate gate timing. In the original approach introduced in [8], the RC interconnect is first approximated by a π-model, and then C_{eff} is computed by equating the charge absorbed by the π-model and C_{eff} up to the delay threshold (i.e., $0.5V_{dd}$). However, a single C_{eff} value focuses primarily on delay estimation and thus cannot accurately approximate slew, which is vital for interconnect analysis [3], as it assumes that the driver output voltage waveform is a linear ramp. Moreover, it ignores the Miller effect and neglects the impact of receiver input capacitance C_p on C_{eff}, delay, and slew estimation. To this end, the authors in [3] introduced a CCS-based methodology to estimate driver output C_{eff} and slew across multiple voltage regions for a π-model load approximation. Nevertheless, the common drawback of both approaches [3,8] is that the π-model reduction of the RC interconnect may induce significant error and runtime overhead in advanced technologies [4].

Other approaches [4,9] model the RC interconnect as a series of connected π-model segments, avoiding a π-model reduction, and iteratively refine C_{eff} (along with delay and slew) on each node i. The latest method introduced in [4] leverages multiple C_{eff} and slew values to enable accurate approximation of the non-linear driver output voltage waveform as a Piecewise Linear (PWL) ramp, which can also be used for accurate interconnect analysis. Therefore, using [4], for each node i of the RC-tree, C_{eff_i} for each region $[\alpha V_{dd} - \beta V_{dd}]$, where $\alpha, \beta \in [0.0, 1.0]$ and $\alpha < \beta$, is calculated by:

$$C^{\alpha-\beta}_{eff_i} = C_i + \sum_{\text{each fanout } j} K^{\alpha-\beta}_j * C_{tot_j}, \text{ where}$$

$$K^{\alpha-\beta}_j = 1 - \frac{X^{\alpha-\beta}_j}{(\beta-\alpha)}\left(e^{-\frac{\alpha}{X^{\alpha-\beta}_j}} - e^{-\frac{\beta}{X^{\alpha-\beta}_j}}\right) \text{ and } X^{\alpha-\beta}_j = \frac{R_{i\to j} C^{\alpha-\beta}_{eff_j}}{S^{\alpha-\beta}_i}. \quad (1)$$

In the above formula, $S^{\alpha-\beta}_i$ is the slew of node i while $C^{\alpha-\beta}_{eff_j}$ is the C_{eff} of fanout node j, corresponding to region $[\alpha V_{dd} - \beta V_{dd}]$. As can be seen, $C^{\alpha-\beta}_{eff_i}$ accounts for a different shielding factor ($K^{\alpha-\beta}_j$) for each fanout node j, which shields C_{tot_j} according to $R_{i\to j}$, $C^{\alpha-\beta}_{eff_j}$, and $S^{\alpha-\beta}_i$. Note that only for receiver input nodes, C_i also incorporates the receiver input pin capacitance (including the Miller component) computed in the corresponding region ($C^{\alpha-\beta}_p$).

2.2. Interconnect timing

SPICE simulation [10,11] is widely regarded as the golden standard for interconnect timing estimation, offering the most accurate and reliable results. However, its computational requirements, particularly in terms of runtime and memory, render it impractical for interconnect timing analysis. In contrast, several analytical moment-based models, such as the Elmore delay and slew, D2M, and S2M, have been proposed for fast timing analysis of interconnects [15,16]. These are based on the first few moments of the RC interconnect impulse response, which, however, assume step inputs and ignore the resistive shielding effect. For each node i of the RC-tree, the jth ($m^{(i)}_j$) moment can be recursively calculated by:

$$m^{(i)}_j = \sum_{\text{each node } k} R_{ki} C_{tot_k} m^{(i)}_{j-1}, \quad \text{with } m^{(i)}_0 = 1, \quad (2)$$

where R_{ki} is the common resistance of paths $0\to i$ and $0\to k$.

By deriving the first and second moments (i.e., m_1 and m_2, respectively), the Elmore delay and slew, D2M, and S2M metrics for each node i of the RC-tree can be calculated as follows:

$$\begin{aligned} \text{Elmore delay}^{(i)} &= m^{(i)}_1, \\ \text{Elmore slew}^{(i)} &= m^{(i)}_1, \\ \text{D2M}^{(i)} &= 0.69\frac{(m^{(i)}_1)^2}{\sqrt{m^{(i)}_2}}, \\ \text{S2M}^{(i)} &= 2.19\sqrt{2m^{(i)}_2 - (m^{(i)}_1)^2} \end{aligned} \quad (3)$$

To account for resistive shielding and the interdependence between slew and C_{eff} (shown in Eq. (1)), the iterative refinement-based methods proposed in [4,9] extend the Elmore delay metric (i.e., Eqs. (2) and (3)), using C_{eff_k} instead of C_{tot_k} on each node k. In this work, we denote this metric as C_{eff}-Elmore, which is computed as follows:

$$\text{C}_{\text{eff}}\text{-Elmore}^{(i)} = \sum_{\text{each node } k} R_{\text{ki}} C_{\text{eff}_k}^{0-50}. \tag{4}$$

Furthermore, the approach of [4] extends the slew metric of [9] for multiple voltage regions. For each node i of the RC-tree, the slew of region $[\alpha V_{\text{dd}} - \beta V_{\text{dd}}]$ is propagated to each fanout node j as follows:

$$S_j^{\alpha-\beta} = \frac{S_i^{\alpha-\beta}}{1 - X_j^{\alpha-\beta}\left(1 - e^{-\frac{1}{X_j^{\alpha-\beta}}}\right)}. \tag{5}$$

Utilizing the above formula, the non-linear voltage waveforms can be approximated using multiple slew values, which enables more accurate interconnect timing analysis compared to moment-based metrics [15, 16].

2.3. Stage timing analysis

Considering the previous discussion, a stage timing analysis methodology has to combine the appropriate gate timing model (e.g., CCS, NLDM), load approximation method (e.g., C_{total}, C_{eff}), and interconnect analysis algorithm to effectively estimate stage delay and slew. The strong gate-interconnect interdependencies, which are evident in Eqs. (1) and (5), necessitate an iterative approach that can incrementally refine the C_{eff}, slew, and delay values on each node of the RC interconnect. Consequently, [4,9] propose an iterative refinement procedure that effectively addresses these issues and accurately estimates stage timing. Fig. 3 outlines the main steps involved in this process:

1. **Driver output waveform estimation:** Given the driver output C_{eff} (initially C_{total}) load approximation, NLDM or CCS driver model LUTs are indexed to compute the output waveform (i.e., driver delay and slew value(s)).
2. **Forward delay and slew propagation:** The driver output waveform is propagated through the RC-tree to each receiver input, utilizing interconnect timing metrics (e.g., Eqs. (3)–(5)) to determine the delay and slew values on each node.
3. **Receiver input pin capacitance calculation:** The propagated receiver input slew values ($S_{\text{rcv}}^{\text{in}}$) are used to access CCS receiver model LUTs and compute the input pin capacitance values (C_p) for all the receivers.
4. **Backward $\mathbf{C_{eff}}$ propagation:** Given the C_p and slew values, C_{eff} values on each node are updated using Eq. (1).

This procedure is repeated until the driver output C_{eff} or slew value(s) converge within a specified threshold. Note that this approach is highly flexible, accommodating different combinations of gate and interconnect algorithms. For example, [9] employs a single C_{eff} and slew value along with NLDM, while [4] utilizes 3 C_{eff} and slew values with CCS to improve accuracy. Alternatively, a more computationally efficient approach might utilize C_{total} with NLDM, which requires only a single iteration of steps 1–2 since C_{total} and the NLDM pin capacitance values are constant. As a result, each stage timing analysis method can achieve a different level of accuracy and efficiency depending on the complexity of the examined stage.

3. Proposed approach

In this section, we present TimeBoost, our ML-driven methodology for optimizing the performance of timing analysis. As shown in Fig. 3, the proposed approach employs XGBoost classifiers to evaluate the complexity of each gate-level stage based on representative stage features. The ML model identifies the most suitable method that effectively exploits the available accuracy and runtime trade-offs, which is then applied by the timing analysis engine during stage delay and slew estimation. Stages with lower complexity can be analyzed using simple yet fast methods, such as C_{total}, moment-based metrics, and NLDM driver and receiver models. In contrast, higher-complexity stages can utilize more accurate but computationally expensive methodologies, such as multiple C_{eff} and slew values, along with CCS driver and receiver models. As a result, gate-level timing analysis can be significantly accelerated without accuracy loss. More specifically, our ML models determine the least computationally expensive method that meets a target accuracy threshold E (e.g., $E < 2\%$). This threshold is based on the mean of stage delay and slew Relative Error (RE) against a golden reference (e.g., SPICE or signoff STA tool) and guides the classifier's decision-making process. If no method meets this requirement, the model defaults to the most accurate available method. Our ML classifiers use five stage timing analysis methods (M1-M5) as target output classes, offering different levels of accuracy and efficiency. As listed in Table 1, the employed methods utilize different gate and interconnect timing analysis techniques, each implemented using the iterative procedure described in Section 2.3. Note that we used Eq. (1) for C_{eff} computation (in single or multiple regions), Eqs. (2) and (3) for all moment-based metrics, Eq. (4) for the C_{eff}-Elmore metric, and Eq. (5) for the slew values of M1 and M2 (in single or multiple regions). Finally, M1 was configured using three voltage regions, i.e., $[0, V_{\text{low}}]$, $[V_{\text{low}}, V_{\text{delay}}]$, and $[V_{\text{delay}}, V_{\text{high}}]$, where V_{low}, V_{delay}, and V_{high} are the lower (e.g., $0.1V_{\text{dd}}$), delay (i.e., $0.5V_{\text{dd}}$), and upper (e.g., $0.9V_{\text{dd}}$) thresholds used to model signal delay and slew, specified in the standard cell library. For all the other methods involving C_{eff} estimation, a single voltage region was used, corresponding to $[0, V_{\text{delay}}]$ for delay estimation and $[V_{\text{low}}, V_{\text{high}}]$ for slew estimation.

Table 1
The output classes of our proposed ML models.

Method	Gate timing	Interconnect timing
M1	CCS and 3 C_{eff}	C_{eff}-Elmore and 3 slew values [4]
M2	NLDM and C_{eff}	C_{eff}-Elmore and single slew value [9]
M3	NLDM and C_{total}	Elmore delay and slew [15]
M4	NLDM and C_{eff}	Elmore delay and slew [15]
M5	NLDM and C_{eff}	D2M and S2M [16]

3.1. Input features

During ML model development, the selection of input features is typically more crucial than the ML algorithm itself. The most critical features exhibit strong correlation with the target output, enabling the generation of accurate and high-performance models that require fewer data points and less training time. Table 2 lists the input features of our proposed classifiers, which focus on capturing the gate, electrical, and topological characteristics of the <driver, RC interconnect, receiver(s)> stage. Firstly, in order to provide information about the driver and its output voltage waveform, we incorporate the cell type (e.g., NAND2), driver input slew, and type of transition at the output. During our experimentation, additional features like the drive strength did not boost prediction accuracy, while incorporating CCS or NLDM LUT information considerably increased inference time and model size. Secondly, we extract key attributes of the RC interconnect, which include minimum, maximum, mean, and variance of the C_i and $R_{i\to j}$ values, as well as C_{total} and R_{total}. Since an interconnect can be of arbitrary size and complexity, these values can compactly capture its electrical properties while considering the resistive shielding effect. Thirdly, we provide information related to the receivers, such as the first 10 C_{tot_i} and $R_{i\to j}$ values along the path to the target receiver, including the respective RC products. Although more segments could be used for long

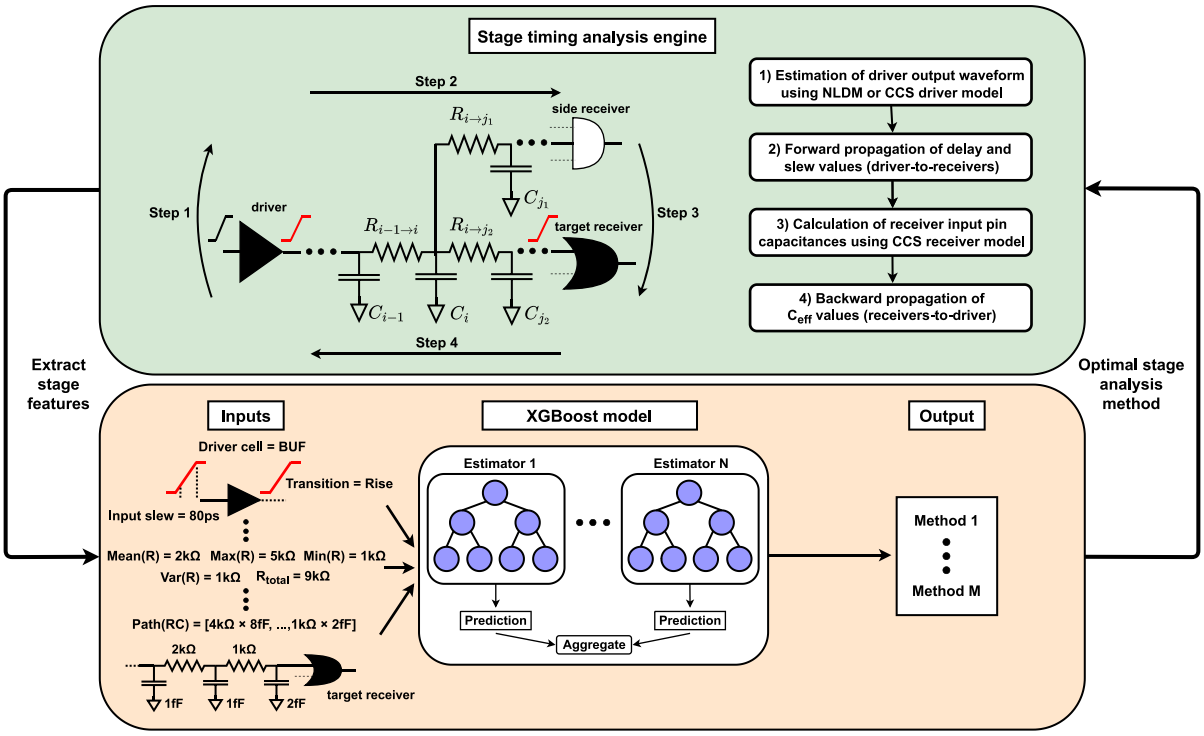


Fig. 3. Proposed ML-driven methodology for optimal stage timing estimation.

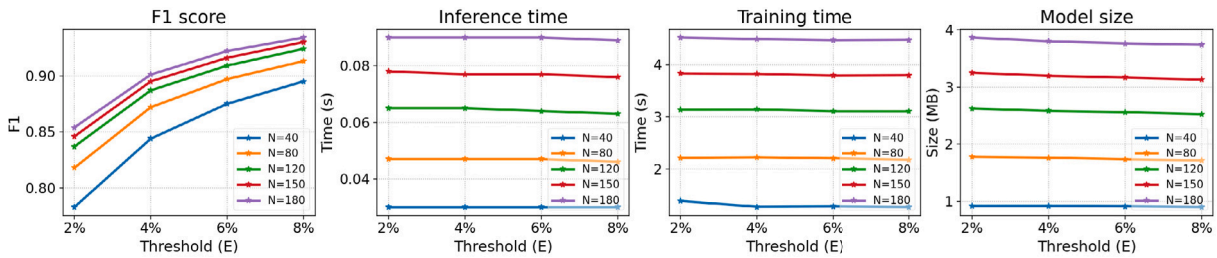
Table 2
The input features of our proposed ML models.

Feature	Size	Description
Driver cell	1	The type of the driver cell
Driver input slew	1	The input slew of the driver cell
Driver output transition	1	The type of transition at the driver cell output (rise or fall)
Min/Max/Mean capacitance	3	Minimum/Maximum/Mean C_i values in the RC graph
Min/Max/Mean resistance	3	Minimum/Maximum/Mean $R_{i\to j}$ values in the RC graph
Total capacitance and resistance	2	C_{total} and R_{total} of the RC graph
Capacitance and resistance variance	2	Standard deviation of C_i and $R_{i\to j}$ values in the RC graph
Path downstream capacitance	10	First 10 C_{tot_i} values of the path to the target receiver
Path resistance	10	First 10 $R_{i\to j}$ values of the path to the target receiver
Path RC product	10	First 10 $R_{i\to j} \times C_{\text{tot}_i}$ values of the path to the target receiver
Target receiver D2M	1	D2M delay value of the target receiver
Fanout	1	Number of receiver cells

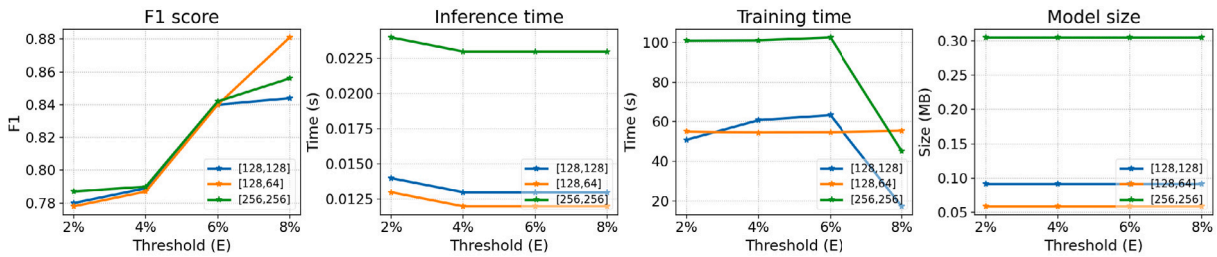
paths, the selected number of values offers a good balance between accuracy and efficiency in terms of ML model performance. Note that for paths consisting of less than ten segments, the remaining path-based values are set to zero to minimize their impact on selecting the optimal method. Moreover, we incorporate the D2M delay value of the target receiver as well as the total number of receivers in the stage (fanout). The extraction of the above features incurs minimal overhead, as most can be directly obtained from the design files (i.e., Verilog, SPEF, Liberty) or computed with a small amount of work. For example, driver-related features are, by definition, inputs to the stage timing analysis engine (e.g., driver input slew, driver cell type), while interconnect or path-based features can be easily computed during the construction and initialization of the RC interconnect graph.

3.2. Model evaluation

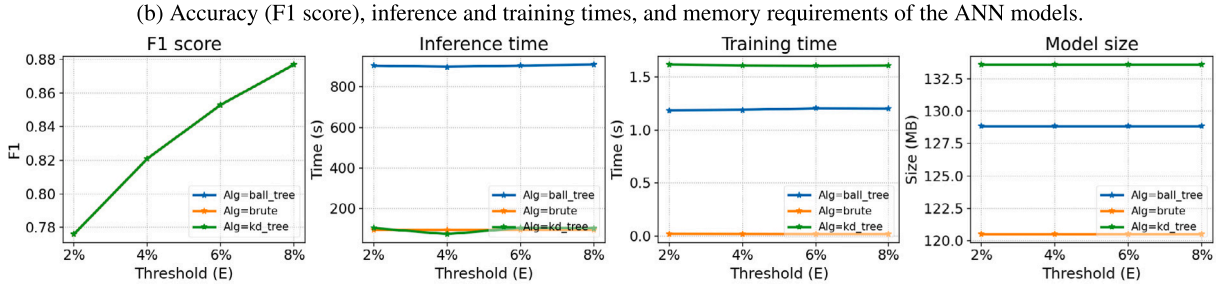
To assess the effectiveness of our feature selection and modeling, we investigated three types of ML models, namely, XGBoost (XGB), Artificial Neural Networks (ANN), and K-Nearest Neighbors (KNN) classifiers. For each model type, we varied different hyperparameters and accuracy thresholds E (2%–8% RE against SPICE), generating multiple models. Specifically, the XGB models were trained using 40–180 estimators (N), ANNs employed three distinct architectures, while KNNs utilized three different algorithms. The ANNs were trained using the Adam optimizer [41], a maximum number of 2000 epochs, and the Rectified Linear Unit (ReLU) activation function for hidden layers and softmax for the output. Note that in this article, the reported architecture refers to the hidden layers used, as the input and output layers depend on the number of input features and output classes, respectively. For example, an ANN with a [128, 128] architecture has an input layer with 45 inputs, two dense hidden layers with 128 neurons, and an output layer with five outputs. The rest of the model hyperparameters for the ANN and XGB models (e.g., batch size for ANNs and leaf samples for XGB) were selected via Bayesian search and optimization. Our dataset consisted of one million stages, with 90% allocated for training and 10% for testing. For each stage in the dataset, we identified the method with the shortest runtime that met the specified threshold E or, if not found, we selected the most accurate one among all five methods. Before training, we normalized all input features using the standard scaler [33] to ensure that no input dominates over the others. Finally, model generation was performed using the XGBoost [35], PyTorch [34], and Sklearn [33] frameworks,



(a) Accuracy (F1 score), inference and training times, and memory requirements of the XGBoost models.



(b) Accuracy (F1 score), inference and training times, and memory requirements of the ANN models.



(c) Accuracy (F1 score), inference and training times, and memory requirements of the KNN models.

Fig. 4. Accuracy (F1 score), inference and training times, and memory requirements of all the evaluated ML models.

utilizing CPU acceleration for KNN and GPU acceleration for XGB/ANN training and inference.[1]

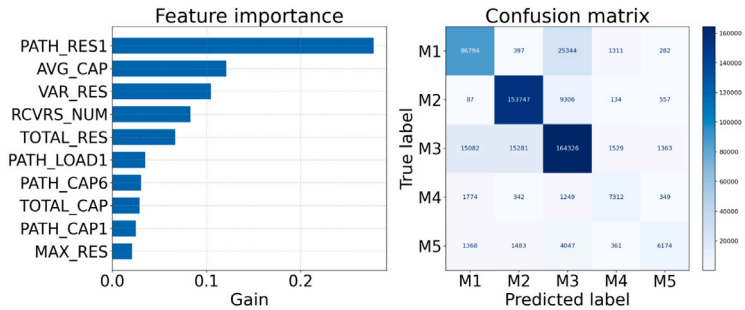
Figs. 4(a)–4(c) illustrate the accuracy (F1 score), training and inference times, and memory requirements of the evaluated ML models. As can be seen, the XGB models outperform ANNs and KNNs, with the more complex (having 120 or more estimators) and those employing higher threshold criteria exhibiting the highest scores (F1 up to 0.934). Increasing the number of estimators enhances predictive performance while targeting a higher accuracy threshold favors selecting the optimal timing analysis method. This highlights the robustness of XGB models, particularly in modeling highly variable, non-continuous spaces using tabular input data. Moreover, Figs. 5(a)–5(b) further demonstrate the impact of our features, showcasing the substantial feature gain of our selection and the excellent classification performance of the XGB models. In comparison, KNNs and ANNs exhibit similar accuracy trends, with accuracy only improving as the threshold E increases, achieving F1 up to 0.853 and 0.881, respectively. As shown in Figs. 4(b)–4(c), the deterministic nature of the evaluated KNN algorithms ensures consistent prediction outcomes, whereas increasing the number of neurons in the ANN hidden layers leads to minor accuracy variations (e.g., [128, 128] vs. [256, 256]). Notably, more complex ANN configurations sometimes performed worse at higher thresholds (i.e., $E < 8\%$) due to overfitting issues and the significant effort required for hyperparameter tuning.

Inference/training times and model size are influenced primarily by the complexity of the underlying model (XGB and ANN) or algorithm (KNN) rather than the accuracy threshold criteria. For instance, increasing the number of estimators in XGB models or the number of neurons in ANNs leads to longer inference and training times, as well as greater disk space requirements for model storage. In the case of KNNs, no actual weights are stored during training. Instead, the entire dataset is memorized, and optionally, an index is generated to access the data points more efficiently, depending on the selected algorithm. During inference, for each input to the model, the algorithm determines the output class based on the k-nearest neighbors. Thus, KNNs are less suitable for optimizing gate-level timing analysis, as they exhibit significantly higher inference time compared to XGB and ANNs. Additionally, model size is highly dependent on the number of features and data points, which scales with the size of the dataset. Conversely, the ANN and XGB models are high-performance, have a low memory footprint, and can utilize both CPU and GPU acceleration. XGB models, in particular, require minimal hyperparameter tuning, can be trained easily in resource-constrained environments, and offer superior accuracy compared to ANNs. Considering these observations, we selected XGB models with 120 estimators for each ML model configuration, as they provide the best balance between accuracy (high F1 score) and performance (low inference/training time).

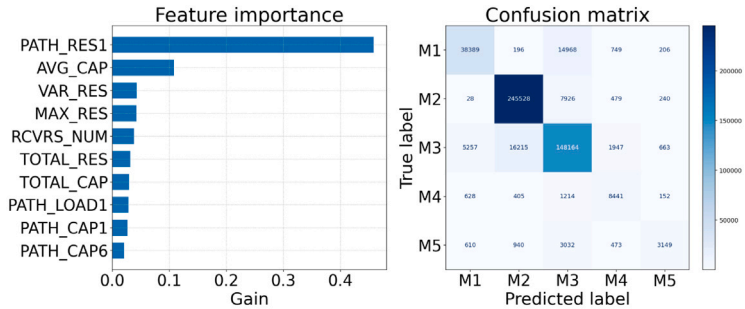
4. Experimental evaluation

To evaluate the effectiveness of our proposed ML-driven approach, we compared it against the base methods listed in Table 1, which also serve as the target output classes. For the ML approach, we implemented two models, ML1 and ML2, targeting 2% and 4% accuracy threshold E, respectively. All base methods, along with the ML models, were integrated into the C++ framework of the ACM TAU 2021 contest [42], which generates representative stages of VLSI circuits and facilitates the evaluation of stage timing analysis methodologies.

[1] More details about the dataset and the experimental setup can be found in Section 4.



(a) Feature gain and confusion matrix for $N = 120$ and $E < 2\%$.



(b) Feature gain and confusion matrix for $N = 120$ and $E < 4\%$.

Fig. 5. Feature gain and confusion matrices of XGB models under different thresholds E.

The framework allows for parallel computation of stage timing using CPU acceleration through multi-threading [43], and the ML models can leverage either CPU or GPU acceleration during inference by setting a single configuration parameter [35]. Specifically, the ML-based methods were evaluated under both acceleration scenarios to assess their performance, while M1-M5 exploit only CPU acceleration. Our experiments were conducted on one million representative stages that cover various driver/receiver combinations and interconnect topologies. The driver and receiver gates are selected from the ASU ASAP 7 nm, Silvaco FreePDK 15 nm, and Silvaco FreePDK 45 nm Process Design Kits (PDKs) [44,45]. The interconnects vary in length and are routed across multiple metal layers, with up to 16 layers available depending on the PDK. Consequently, the RC values of the interconnect models span a wide range, determined by the resistance and capacitance of the selected metal layers and the resistance of the connecting vias. Moreover, the stages are generated so that driver input slew ($S^{\text{in}}_{\text{drv}}$), driver output effective capacitance (C_{eff}), and receiver output capacitance ($C^{\text{out}}_{\text{rcv}}$) cover the entire range of the precharacterized LUTs and even beyond that. The golden stage delay and slew values were obtained using the Xyce Parallel Electronic Simulator [10]. Finally, all experiments were run on a Linux workstation, equipped with a 3.70 GHz 12-thread Ryzen 5600X CPU, an NVIDIA RTX 3700 GPU, and 16 GB of RAM.

Table 3 presents the mean stage delay and slew RE against SPICE, the runtime, and the speedup over the most accurate method (M1) achieved by each approach. Firstly, it is evident that accuracy decreases as we transition from the most complex multiple C_{eff}-based (M1) to the simplest C_{total}-based (M3) method. Secondly, moment-based methods (M3-M5) generally exhibit lower accuracy, with M4-M5 being rarely selected by the ML models, as depicted in the confusion matrices of Fig. 5. In contrast, our ML approaches achieve accuracy in line with their respective thresholds and our available methods. ML1 closely follows the best available method (M1), exhibiting stage delay and slew RE of 2.51% and 2.85%, respectively. Moreover, ML2 effectively maintains errors well below the target 4% threshold, achieving 3.39% and 3.87% stage delay and slew RE. Note that ML1 and ML2 achieve the same accuracy when utilizing CPU or GPU acceleration, as it does not affect accuracy, but only runtime performance.

Considering the runtime results, we can observe a clear correlation between the complexity of each base method and its respective performance. In particular, M1 exhibits the highest runtime (12.5 s), whereas the simplest M3 method records the lowest (0.55 s). Moreover, our proposed ML approaches, ML1 and ML2, achieve substantial speedups compared to the most accurate method (M1) while maintaining high accuracy levels. With CPU-accelerated inference, ML1 and ML2 expedite timing analysis by 2.29× and 3.28×, respectively. When leveraging GPU acceleration, the speedups increase to 2.62× and 3.86×, respectively, representing a 15% improvement over the CPU-accelerated implementation. Furthermore, the overall overhead introduced by the ML model to timing analysis is reduced to 3%–4%, compared to 8%–10% corresponding to CPU-accelerated execution. Therefore, when GPU hardware is available, the resulting performance gains can expedite timing analysis without compromising predictive accuracy or ease of use. It is crucial to note that adopting more resource-intensive signoff-accurate methods, such as AWE and RICE [13,14], as output classes of our ML models would further amplify the performance and improve the accuracy of the proposed approach. Since computationally expensive methods demand longer runtimes, selecting more efficient alternatives yields greater relative speedups, as model inference represents a constant yet smaller cost compared to the total stage timing analysis time. Thus, this further enhances the practicality and effectiveness of our ML models, particularly in performance-critical applications like timing-driven iterative optimization flows [1].

5. Conclusions

In this work, we presented TimeBoost, a simple yet powerful learning-based approach to accelerate timing analysis. The proposed method employs XGBoost classifiers, which, given representative features of a gate-level stage, predict the most suitable timing analysis method, exploiting the available accuracy/runtime trade-offs. Experimental results demonstrate that our ML-driven approach delivers a 3.86× speedup over our best available stage timing analysis method while maintaining commendable accuracy.

Table 3
Accuracy and runtime results of our ML-driven approach against different stage timing analysis methodologies.

Method		Mean RE against SPICE		Runtime	
		Stage delay	Stage slew	Time (s)	Speed-up
M1		2.25%	2.77%	12.5	1×
M2		4.43%	4.57%	3.09	4.05×
M3		9.15%	9.58%	0.55	22.73×
M4		5.9%	6.82%	3.76	3.33×
M5		5.73%	6.64%	3.85	3.25×
ML1	CPU	2.51%	2.85%	5.45	2.29×
	GPU	2.51%	2.85%	4.78	2.62×
ML2	CPU	3.39%	3.87%	3.81	3.28×
	GPU	3.39%	3.87%	3.24	3.86×

CRediT authorship contribution statement

Anastasis Vagenas: Writing – original draft, Software, Methodology, Data curation, Conceptualization. **Dimitrios Garyfallou:** Writing – original draft, Software, Methodology, Data curation, Conceptualization. **Nestor Evmorfopoulos:** Writing – original draft. **George Stamoulis:** Writing – original draft.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

A preliminary version of this work has been published in SMACD 2024 Conference Proceedings [38]. This work is supported by the Center of Research Innovation and Excellence (CRIE) of the University of Thessaly, Greece.

Data availability

Data will be made available on request.

References

[1] Bhasker J, Chadha R. Static timing analysis for nanometer designs: A practical approach. Springer; 2009.
[2] Goering R. What's needed to "fix" timing signoff? In: Panel of the 50th Design Automation Conference. DAC, 2013.
[3] Garyfallou D, et al. Gate Delay Estimation With Library Compatible Current Source Models and Effective Capacitance. IEEE Trans. on Very Large Scale Integration Systems 2021;29(5):962–72. http://dx.doi.org/10.1109/TVLSI.2021.3061484.
[4] Garyfallou D, et al. Leveraging Machine Learning for Gate-level Timing Estimation Using Current Source Models and Effective Capacitance. In: Proc. of the 22nd Great Lakes Symposium on VLSI. GLSVLSI, 2022, p. 77–83. http://dx.doi.org/10.1145/3526241.3530343.
[5] Darmi M, Cherif L, Benallal J, Elgouri R, Hmina N. Integrated Circuit Conception: A Wire Optimization Technic Reducing Interconnection Delay in Advanced Technology Nodes. Electronics 2017;6(4). http://dx.doi.org/10.3390/electronics6040078.
[6] Garyfallou D, et al. Accurate Estimation of Dynamic Timing Slacks using Event-Driven Simulation. In: Proc. of the 21st International Symposium on Quality Electronic Design. ISQED, 2020, p. 225–30. http://dx.doi.org/10.1109/ISQED48828.2020.9137017.
[7] Synopsys - Composite Current Source (CCS), [Online]. Available: http://www.opensourceliberty.org/ccspaper/. [Accessed 12 June 2025].
[8] Qian J, Pullela S, Pillage L. Modeling the "Effective Capacitance" for the RC Interconnect of CMOS Gates. IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems 1994;13(12):1526–35. http://dx.doi.org/10.1109/43.331409.
[9] Puri R, Kung DS, Drumm AD. Fast and accurate wire delay estimation for physical synthesis of large ASICs. In: Proc. of the 12th Great Lakes Symposium on VLSI. GLSVLSI, 2002, p. 30–6. http://dx.doi.org/10.1145/505306.505314.
[10] Keiter E, et al. Xyce parallel electronic simulator users' guide version 7.6. Tech. rep., Sandia National Lab. (SNL-NM); 2022.
[11] Synopsys - HSPICE®, [Online]. Available: https://www.synopsys.com/verification/ams-verification/hspice.html. [Accessed 12 June 2025].
[12] Antoniadis C, Evmorfopoulos N, Stamoulis G. Efficient Sparsification of Dense Circuit Matrices in Model Order Reduction. In: Proc. of the 24th Asia South Pacific Design Automation Conference. ASP-DAC, 2019, p. 255–60. http://dx.doi.org/10.1145/3287624.3287658.
[13] Pillage LT, Rohrer RA. Asymptotic Waveform Evaluation for Timing Analysis. IEEE Trans. on Computer-Aided Design of Circuits and Systems 1990;9(4):352–66. http://dx.doi.org/10.1109/43.45867.
[14] Ratzlaff CL, Pillage LT. RICE: rapid interconnect circuit evaluation using AWE. IEEE Trans. on Computer-Aided Design of Circuits and Systems 1994;13(6):763–76. http://dx.doi.org/10.1109/43.285250.
[15] Bakoglu HB. Circuits, interconnections, and packaging for VLSI. Addison-Wesley Pub. Co.; 1990.
[16] Agarwal K, Sylvester D, Blaauw DT. Simple metrics for slew rate of RC circuits based on two circuit moments. In: Proc. of the 40th Design Automation Conference. DAC, 2003, p. 950–3. http://dx.doi.org/10.1145/775832.776070.
[17] Liu F, Kashyap CV, Alpert CJ. A delay metric for RC circuits based on the Weibull distribution. IEEE Trans. on Computer-Aided Design of Circuits and Systems 2004;23(3):443–7. http://dx.doi.org/10.1109/TCAD.2004.823343.
[18] Kashyap C, Alpert C, Liu F, Devgan A. Closed-form expressions for extending step delay and slew metrics to ramp inputs for RC trees. IEEE Trans. on Computer-Aided Design of Circuits and Systems 2004;23(4):24–31. http://dx.doi.org/10.1109/TCAD.2004.825861.
[19] Jiang IH-R. Lightning Talk: All Routes to Timing Closure. In: Proc. of the 60th Design Automation Conference. DAC, 2023, p. 1–2. http://dx.doi.org/10.1109/DAC56929.2023.10247801.
[20] Guo Z, Huang T-W, Lin Y. Accelerating Static Timing Analysis Using CPU-GPU Heterogeneous Parallelism. IEEE Trans. on Computer-Aided Design of Circuits and Systems 2023;42(12):4973–84. http://dx.doi.org/10.1109/TCAD.2023.3286261.
[21] Guo G, Huang T-W, Lin Y, Wong M. GPU-accelerated Path-based Timing Analysis. In: Proc. of the 58th Design Automation Conference. DAC, 2021, p. 721–6. http://dx.doi.org/10.1109/DAC18074.2021.9586316.
[22] Lin S, Guo G, Huang T-W, Sheng W, Young E, Wong M. GCS-Timer: GPU-Accelerated Current Source Model Based Static Timing Analysis. In: Proc. of the 61st Design Automation Conference. DAC, 2024, p. 1–6. http://dx.doi.org/10.1145/3649329.3655983.
[23] Huang G, et al. Machine Learning for Electronic Design Automation: A Survey. ACM Trans. on Design Automation of Electronic Systems 2021;26(5):1–46. http://dx.doi.org/10.1145/3451179.
[24] Yu B. Machine Learning in EDA: When and How. In: Proc. of the 5th workshop on Machine Learning for CAD. MLCAD, 2023, p. 1–6. http://dx.doi.org/10.1109/MLCAD58807.2023.10299822.
[25] Abrishami MS, Pedram M, Nazarian S. CSM-NN: Current Source Model Based Logic Circuit Simulation - A Neural Network Approach. In: Proc. of the International Conference on Computer Design. ICCD, 2019, p. 393–400. http://dx.doi.org/10.1109/ICCD46524.2019.00061.
[26] Ebrahimipour SM, Ghavami B, Mousavi H, Raji M, Fang Z, Shannon L. Aadam: A Fast, Accurate, and Versatile Aging-Aware Cell Library Delay Model using Feed-Forward Neural Network. In: Proc. of the 39th International Conference on Computer-Aided Design. ICCAD, 2020, p. 1–9. http://dx.doi.org/10.1145/3400302.3415605.
[27] Vagenas A, Garyfallou D, Evmorfopoulos N, Stamoulis G. Advanced gate-level glitch modeling using ANNs. In: Proc. of the 61st Design Automation Conference. DAC, 2024, p. 1–6. http://dx.doi.org/10.1145/3649329.3657351.
[28] Chen Y, Guo Z, Wang R, Huang R, Lin Y, Zhuo C. Dynamic Supply Noise Aware Timing Analysis With JIT Machine Learning Integration. IEEE Trans. on Computer-Aided Design of Circuits and Systems 2024;43(5):1511–24. http://dx.doi.org/10.1109/TCAD.2023.3342603.
[29] Kahng A, Kang S, Lee H, Nath S, Wadhwani J. Learning-based approximation of interconnect delay and slew in signoff timing tools. In: Proc. of the International workshop on System Level Interconnect Prediction. SLIP, 2013, p. 1–8. http://dx.doi.org/10.1109/SLIP.2013.6681682.
[30] Barboza EC, Shukla N, Chen Y, Hu J. Machine Learning-Based Pre-Routing Timing Prediction with Reduced Pessimism. In: Proc. of the 56th Design Automation Conference. DAC, 2019, p. 1–6. http://dx.doi.org/10.1145/3316781.3317857.
[31] Zhu L, Gu Y, Guo X. RC-GNN: Fast and Accurate Signoff Wire Delay Estimation with Customized Graph Neural Networks. In: Proc. of the 5th International Conference on Artificial Intelligence Circuits and Systems. AICAS, 2023, p. 1–5. http://dx.doi.org/10.1109/AICAS57966.2023.10168562.
[32] Liu H, Wu S, Tao S, Xie B, Li X, Li G. Accurate Timing Path Delay Learning Using Feature Enhancer with Effective Capacitance. In: Proc. of the International Symposium of Electronics Design Automation. ISEDA, 2023, p. 280–5. http://dx.doi.org/10.1109/ISEDA59274.2023.10218429.

[33] Pedregosa F, et al. Scikit-learn: Machine Learning in Python. J Mach Learn Res 2011;12(85):2825–30.
[34] Paszke A, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019, arXiv preprint arXiv:1912.01703.
[35] Chen T, Guestrin C. XGBoost: A Scalable Tree Boosting System. In: Proc. of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data mining. KDD, 2016, p. 785–94. http://dx.doi.org/10.1145/2939672.2939785.
[36] UTH-Timer, [Online]. Available: https://github.com/digaryfa/UTH-Timer. [Accessed 12 June 2025].
[37] Abbaspour S, Pedram M, Ajami A, Kashyap C. Fast Interconnect and Gate Timing Analysis for Performance Optimization. IEEE Trans. on Very Large Scale Integration Systems 2006;14(12):1383–8. http://dx.doi.org/10.1109/TVLSI.2006.887834.
[38] Vagenas A, Garyfallou D, Evmorfopoulos N, Stamoulis G. A learning-based method for performance optimization of timing analysis. In: Proc. of the 20th International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design. SMACD, 2024, p. 1–4. http://dx.doi.org/10.1109/SMACD61181.2024.10745408.
[39] Synopsys - PrimeLib®, [Online]. Available: https://www.synopsys.com/implementation-and-signoff/signoff/primelib.html. [Accessed 12 June 2025].
[40] Cadence - Liberate™ Trio™ Characterization Suite, [Online]. Available: https://www.cadence.com/en_US/home/tools/custom-ic-analog-rf-design/library-characterization/liberate-trio-characterization-suite.html. [Accessed 12 June 2025].
[41] Kingma DP, Ba J. Adam: A method for stochastic optimization. 2014, arXiv preprint arXiv:1412.6980.
[42] TAU 2021 Timing Contest - Stage Delay Calculator Using Current Source Models, [Online]. Available: https://sites.google.com/view/tau-contest-2021/. [Accessed 12 June 2025].
[43] The OpenMP API specification for parallel programming, [Online]. Available: http://www.openmp.org/. [Accessed 12 June 2025].
[44] ASU - ASAP7 PDK, [Online]. Available: https://github.com/The-OpenROAD-Project/asap7/. [Accessed 16 April 2025].
[45] Silvaco - Open-Cell 15 nm and 45 nm FreePDK Libraries, [Online]. Available: https://si2.org/open-cell-library/. [Accessed 12 June 2025].

Anastasis Vagenas received his M.Eng. from the Department of Electrical and Computer Engineering, University of Thessaly, Volos, Greece, in 2022. He is currently a Ph.D. candidate in the same department and focuses on Machine Learning techniques for efficient optimization and modeling in timing analysis. His research interests include timing and power analysis, signal integrity, and standard cell pre-characterization.

Dimitrios Garyfallou is a Postdoctoral Researcher and Adjunct Lecturer at the Department of Electrical and Computer Engineering of the University of Thessaly, Greece. He earned his Diploma, M.Sc., and Ph.D. from the same department in 2014, 2015, and 2021, respectively. His research focuses on algorithms and machine learning techniques for Electronic Design Automation (EDA), with emphasis on timing and power analysis, circuit reliability, and large-scale simulation. He was a Research Assistant at the Center for Research and Technology - Hellas (CERTH) in 2015 and a Visiting Research Associate at Queen's University Belfast, U.K., in 2017 and 2019. Dimitrios has authored over 23 articles in international journals and conferences, received 2 best paper award nominations, served on technical and organizing committees for international conferences, and reviewed for leading journals. He has participated in 13 EU, national, and industry-funded research projects and founded an EDA startup. Dr. Garyfallou has received M.Sc. and Ph.D. fellowships from the Center of Research, Innovation, and Excellence (CRIE) of the University of Thessaly, the HiPEAC collaboration grant in 2017, and the first place award in the ACM TAU 2020 and 2021 timing analysis contests.

Nestor Evmorfopoulos is an Associate Professor at the Department of Electrical and Computer Engineering of the University of Thessaly, Greece. He received the B.E., M.Sc., and Ph.D. degrees in Electrical and Computer Engineering from the National Technical University of Athens. His main research interests are focused on mathematical and computational methods for CAD tools and algorithms for integrated circuit design, with emphasis on large-scale circuit, thermal, electromagnetic and physical simulation. He has published over 100 articles in international journals and conferences and holds 5 U.S. patents. He received the "William J. McCalla" Award for the best paper at the International Conference on Computer-Aided Design in 2021 and was nominated for the same award in 2012, while he has also received 1 best paper award and 4 best paper nominations in other international conferences. He serves at the Scientific Committees of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD) and the IEEE/ACM Design Automation Conference (DAC), while in 2024 he served as General Chair of the IEEE International Conference on Synthesis, Modeling, Analysis and Simulation Methods, and Applications to Circuit Design (SMACD). In 2020 he received a Fulbright award for conducting research at the University of Minnesota, USA, as a Visiting Scholar.

Georgios (George) Stamoulis received his Diploma from the National Technical University of Athens in 1989. He was awarded the M.S. (1991) and Ph.D. (1994) degrees from the University of Illinois at Urbana-Champaign. After one year as a Visiting Assistant Professor at the University of Iowa, he joined Intel Corp. In 2001 he was elected Assistant Professor at the Department of Electronic and Computer Engineering at the Technical University of Crete. In 2003 he became an Associate Professor at the Department of Computer and Telecommunications Engineering at the University of Thessaly. In 2009 he became Professor. From 2013 to 2018 he was the Head of the newly formed Computer Science Department and from 2018 to June 2023 the Dean of the School of Engineering of the University of Thessaly. Since 2023 he is a Member of the Management Council of the University of Thessaly. His research interests focus on the analysis and optimization of average and maximum power of integrated circuits, the analysis and optimization of the maximum voltage drop on the power supply lines of integrated circuits, low power design, reliability analysis and optimization, and the application of massively parallel and deep-learning techniques to the aforementioned problems. He has founded two startups, authored over 250 papers in journals and major conferences, claims three US patents and has over 2500 citations to his work.